Projet Python - Logs

Le code :

Création d'utilisateur :

Person : Classe objet :

Person.py | User.py | main.py | password_utils.py

ProjetPythonT1 > classes > Person.py > Person > set_nom

```python
class Person(object):
    """La classe Person pour un utilisateur"""

    def __init__(self, nom, prenom):
        print("Initialisation de la classe Person avec comme paramètre le nom et le prénom")
        self.nom = nom
        self.prenom = prenom

    def get_nom(self):
        return self.nom

    def get_prenom(self):
        return self.prenom

    def set_nom(self, nouveau_nom):
        if nouveau_nom == "":
            print ("Erreur : Il faut compléter le champ.")
        else:
            self.nom = nouveau_nom
            print("Le nom a été modifié.")

    def set_prenom(self, nouveau_prenom):
        if nouveau_prenom == "":
            print("Nop il manque le prenom")
        else:
            self.prenom = nouveau_prenom
            print("C'est bon")

    def afficher(self):
        print(f"Personne : {self.nom}, {self.prenom}")
```

On a fait l'initialisation avec le nom et le prenom.

(self parce qu'on va le réutiliser)

get_nom pour récupérer le nom avec la méthode getter

pareil pour le prenom

set_nom avec nouveau_nom en parametres, et dedans on s'assure qu'au moins un caractère est rentré.

Pareil pour le prenom

On fait une méthode afficher, et ça permet d'afficher le nom et prenom via un print(f""), un f-string.

User : Hérite de la classe objet Person :

```
from classes.Person import Person
from utils.password_utils import generer_mdp, hash_mdp

class User(Person):
    """Classe qui hérite de Person, et donc qui récupère nom et prenom ains

    def __init__(self, nom, prenom):
        print("Création d'un utilisateur...")
        Person.__init__(self, nom, prenom)
        self.login = self.generer_login(nom, prenom)

        mdp_clair = generer_mdp()
        print(f"Mot de passe généré : {mdp_clair}")
        self.password = hash_mdp(mdp_clair)

    def generer_login(self, nom, prenom):
        """Generation du login via le nom et prenom"""
        premiere_lettre = prenom[0]
        login = premiere_lettre + nom
        login = login.lower() # ca met en minuscule ( avec sananes, ça fais
        return login
```

On importe Person du dossier classes

On importe les méthodes generer_mdp et hash_mdp de password_utils qui est dans le dossier utils

Donc on fait en sorte que la classe User hérite de Person pour récuperer le nom et le prénom.

On initialise ici le nom et prénom qui vient de la classe Person

```
Person.__init__(self, nom, prenom)
```

Ensuite on définit login pour dire que le nom et prenom récupérés seront dans self.login parce qu’on va les réutiliser.

On génère un mdp en clair via la méthode generer_mdp()

et on définit le password (mot de passe final) en utilisant la méthode permettant de hashé le mdp qui est en clair.

On définit ensuite une méthode pour générer le login avec les contraintes demandées : première lettre du prénom + nom de famille

0 parce qu’il s’agit de la première lettre + le nom

Et on s’assure que cela soit en minuscule.

On retourne le résultat lors de l’appel de la méthode.

Afin de s’assurer qu’il n’y ait pas de doublons, nous allons modifier la méthode de création du login :

```python
from classes.Person import Person
from utils.password_utils import generer_mdp, hash_mdp
from utils.csv_manager import charger_users

class User(Person):
    """Classe qui hérite de Person, et donc qui récupère nom et prenom ainsi que les r

    def __init__(self, nom, prenom):
        print("Création d'un utilisateur...")
        Person.__init__(self, nom, prenom)
        self.login = self.generer_login(nom, prenom)

        mdp_clair = generer_mdp()
        print(f"Mot de passe généré : {mdp_clair}")
        self.password = hash_mdp(mdp_clair)

    def generer_login(self, nom, prenom):
        """Generation du login via le nom et prenom"""
        premiere_lettre = prenom[0]
        login_base = premiere_lettre + nom
        login_base = login_base.lower() # ca met en minuscule ( avec sananes, ça faisa

        print("Chargement des utilisateurs...")
        utilisateurs = charger_users()
        print(f"Nombre de users : {len(utilisateurs)}")

        login_exist = []
        for user in utilisateurs:
            login_exist.append(user['login'])
        print(f"Logins existant : {login_exist}")

        login = login_base
        compteur = 1

        while login in login_exist:
            login = login_base + str(compteur) # galère : str va convertir le nombre e
            compteur = compteur + 1

        print(f"Login final : {login}")
        return login
```

Explication : Avant de return login, on fait en sorte qu'il y ait une vérification.

On créé une table login_exist, où on ajoute le login du user (avec append)

Pour chaque user (variable définit ici) dans utilisateurs (qui prends en valeur la méthode charger_users, afin de récupérer tous les users), on ajoute le login dans login_existant. En gros, on extrait uniquement le login des informations enregistrer avec la méthode.

avec login = login_base, on vérifie si la chaine de caracteres est la même, on définit compteur à 1 et lorsque login est dans login_exist, alors login vaut : login_base + le caractere (compteur). (Le caractere sinon ça bug pcq int + string c'est bug)

Génération du mot de passe : password_utils

```python
import random
import string
import hashlib

def generer_mdp(taille = 12): # définition d'une valeur par défaut comme ça
    """Generation d'un mdp aléatoire avec la taille x"""
    caracteres = string.ascii_letters + string.digits + string.punctuation

    mdp = ""
    for i in range(taille):
        mdp += caracteres[random.randint(0, len(caracteres)-1)]

    return mdp

def hash_mdp(mdp):
    """Hash le mdp avec l'algorithme SHA-256"""
    mdp_hashed = hashlib.sha256(mdp.encode()).hexdigest() # .encode en byte
    return mdp_hashed
```

On importe les librairies random, string et hashlib :

random pour la récupération “aléatoire” des caractères.

String pour les différents types de caractères.

hashlib pour l’encodage et le hashage via l’algorithme SHA-256.

La méthode generer_mdp avec en valeur par défaut la taille qui fait 12.

On définit la variable caracteres qui prends toutes les lettres de la table ASCII, les chiffres et les ponctuations.

On définit ensuite mdp vide, et on y met une boucle qui tourne selon la taille (définit sinon 12), qui récupère alors un élément de la variable caracteres, en commençant par 0 et dont la taille fait -1 puisqu’on commence à 0.

Pour le hashage :

hash_mdp, qui prends la variable mdp, et qui applique l’algorithme SHA-256. (avec l’encodage) et après on hexdigest, qui rends en hexadécimales les valeurs encodées en bytes, pour que ça soit plus digeste pour nous, humains.

On retourne alors le résultat dans mdp_hashed.

Mise en place du système de droits : Admin

classes > Admin.py > Admin > verification_gerer_users

```python
from classes.User import User

class Admin(User):
    """Classe héritant de User, qui hérite lui meme de Person, pour eviter de refair

    def __init__(self, nom, prenom, niveau_droits=1):
        print("Création d'un admin...")
        User.__init__(self, nom, prenom)
        self.niveau_droits = niveau_droits

    def afficher_admin(self):
        """Affiche les infos de l'admin"""
        print(f"Admin : {self.nom} {self.prenom}")
        print(f"Login : {self.login}")
        print(f"Niveau de droits : {self.niveau_droits}")

    def verification_gerer_users(self):
        """Verifie si l'admin peut gérer les users"""
        return self.niveau_droits >=1

    def verification_gerer_admins(self):
        """Verifie si le superadmin peut gérer les autres admins"""
        return self.niveau_droits >=2
```

On récupère les informations contenues de la classe User qui est dans le dossier classes

On définit la classe Admin ayant hérité de User.

Les attributs du constructeur : le nom, le prenom et on rajoute un attribut : le niveau_droits, que l'on met par défaut à 1

On définit la méthode afficher_admin, qui servira à afficher les informations de l'administrateur.

On définit verification_gerer_users pour vérifier les droits (donc ici admin), et on définit verification_gerer_admins pour vérifier les droits encore + haut. (pour superadmin)

Du coup, on retourne un booléen pour savoir si le niveau de droits attribué au compte puisse oui ou non gérer tel ou tel cas.

Le stockage d'informations :

main.py | users.csv | Person.py | User.py | password_utils.py | Admin.py | csv_manager.py

C:\Users\RyanZ\Desktop\ProjetPythonT1\main.py

```python
import csv
import os # Interaction avec le système d'exploitation. Il permet de vérifier le fichier avec os.path, et de créer un dossier avec makedirs

FICHIER_USERS = "data/users.csv"

def init_fichiers():
    """Initialise le fichier en CSV, et si le fichier n'existe pas, il le créé"""
    if not os.path.exists("data"):
        os.makedirs("data")

    if not os.path.exists(FICHIER_USERS):
        with open(FICHIER_USERS, 'w', newline='', encoding='utf-8') as fichier:
            writer = csv.writer(fichier)
            writer.writerow(['nom', 'prenom', 'login', 'password', 'type', 'niveau_droits']) # '' et "" agissent pareil, c'est pour être clair dans le code
        print(f"fichier {FICHIER_USERS} créé")

def save_user(user, type_user):
    """Save le user dans le fichier csv"""

    init_fichiers()

    with open(FICHIER_USERS, 'a', newline='', encoding='utf-8') as fichier: # a c'est pour ajouter, alors que w c'est pour supprimé et réécrire
        writer = csv.writer(fichier)
        if type_user == "admin":
            niveau = user.niveau_droits
        else:
            niveau = 0
            writer.writerow([user.nom, user.prenom, user.login, user.password, type_user, niveau])

    print(f"Le compte {type_user.capitalize()} {user.login} est sauvegardé")

def charger_users():
    """Charger tous les users du fichier csv"""
    init_fichiers()
    users = []
    with open(FICHIER_USERS, 'r', encoding='utf-8') as fichier: # r pour la lecture
        reader = csv.DictReader(fichier)
        for i in reader: # pour chaque ligne
            users.append(i) # append = ajouter
    return users
```

On utilise la librairie csv (format du fichier de stockage) et la librairie os afin d'interagir avec le système d'exploitation. La méthode os.path va permettre de vérifier le fichier (s'il existe) et le makedirs va le créer.

On met le chemin vers là où l'on veut le fichier.csv dans la variable FICHIER_USERS.

On definit la méthode init_fichiers() qui est un constructeur pour :

- Créer le dossier data s'il n'existe pas, alors on affiche création du dossier, et on créé le dossier
- Créer le fichier users.csv s'il n'existe pas dans data. On le créé, puis on ouvre en droit d'écriture le chemin data/users.csv, newline='' pour le saut de ligne, avec utf-8 pour gérer les spécificités du clavier français, et tout ça dans la variable appelée fichier.
- On met la variable fichier en CSV, puis on met le résultat dans writer.
- writerow écrit une ligne dans le CSV, et range les données reçues selon : nom, prenom, login, password, type et niveau_droits.

save_user() est une méthode qui sert à enregistrer le user dans le fichier csv, l'ajouter :

- Dedans, on aura le user (Admin ou User) à sauvegarder, et le type_user pour que ça soit un user ou un admin.
- On utilise la méthode init_fichiers () pour être sûr que tout existent avant d'ajouter des informations.
- On ouvre le fichier en mode append pour ajouter, si on le fait en écriture, il effacera tout avant d'écrire des éléments.
- On remet le mode CSV, puis on ajoute un niveau de droits. Si c'est un admin, alors on récupère son niveau_droits. Sinon, c'est un user normal. (niveau 0, pas de droits admin ou +)
- writer.writerow : on écrit une nouvelle ligne dans le CSV en récupérant les informations de user, et on affiche tout ça, en mettant la première lettre en majuscule (capitalize)

charger_users() est une méthode afin de charger toutes les informations dans le fichier CSV :

- On créé une table vide, où l'on y met la liste de tous les users.
- On utilise cette fois le mode de lecture lorsqu'on ouvre le fichier.
- DictReader nous simplifie beaucoup l'organisation pour la lecture des données. Pour chaque ligne, on ajoute i, donc on lit le fichier.

L'authentification :

Pour pouvoir coder l'authentification, il faut bien définir avant le système d'authentification. Nous avons donc décidé de faire ainsi :

1 - Le user tape son login et son mdp

2 - On cherche l'utilisateur avec le login entré

3 - On hashe le mdp entré

4 - On compare le hash avec celui qui est stocké

5 - Si ça correspond, on confirme la connexion

5bis - Si ça ne correspond pas, alors on met un message d'erreur.

L'authentification se fait dans le fichier Auth.py :

```python
from utils.csv_manager import charger_users
from utils.password_utils import hash_mdp

def authentifier(login, mot_de_passe):
    """Authentification via le login et le mot de passe"""

    utilisateurs = charger_users()
    print(f"Voici le nombre d'utilisateurs : {len(utilisateurs)}")

    for user in utilisateurs:
        if user['login'] == login:
            print(f"L'identifiant est : {login}")
            mdp_hash = hash_mdp(mot_de_passe)
            print(f"DEBUG: Hash du mdp tapé : {mdp_hash}")
            print(f"DEBUG: Hash stocké : {user['password']}")
            print("Verification du mot de passe...")

            if mdp_hash == user['password']:
                print("Connexion réussie !")
                return user
            else:
                print("Connexion échouée")
                return None

    print(f"Login introuvable.")
    return None
```

On utilise charger_users et hash_mdp qui viennent respectivement de csv_manager, et de password_utils.

On définit la méthode authentifier avec comme paramètres le login et le mot de passe.

On met dans la variable utilisateurs la méthode charger_users afin de récupérer toutes les informations du fichier csv, et on affiche ça dans le print avec la méthode len qui permet de compter le nombre d'objet d'un dictionnaire.

On cherche l'utilisateur avec le login.

Pour chaque user dans utilisateurs (qui charge tous les users), si le login est le même, alors on affiche : L'identifiant est :

Ensuite, on hash le mot de passe entré, et on compare le avec le if mdp_hash == user['password']:

Si c'est bon, on retourne le user.

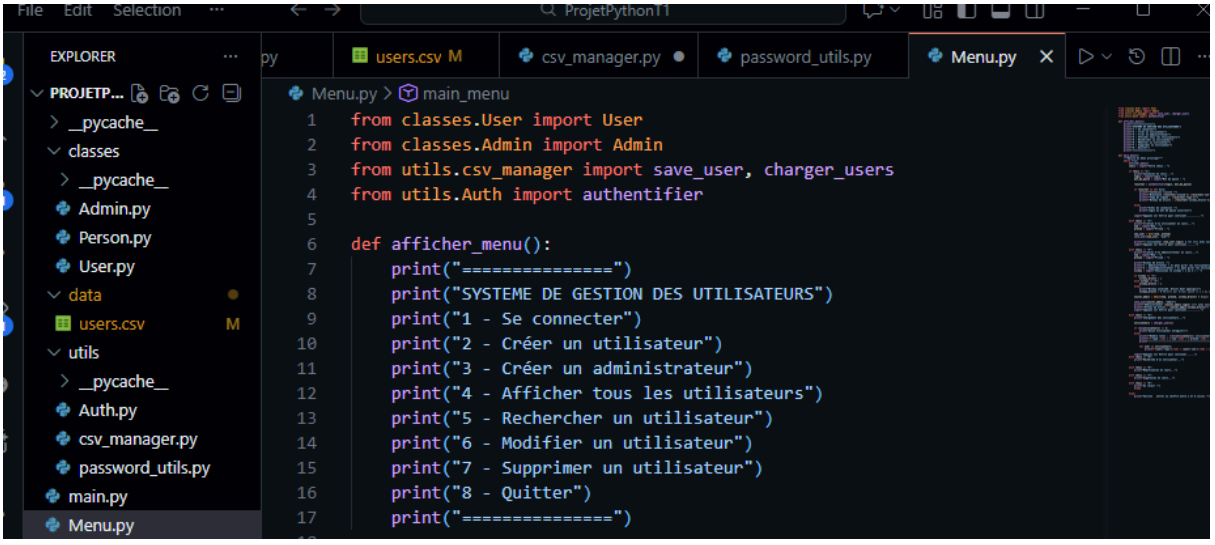
Sinon, on affiche connexion échouée, et on retourne None.

Menu.py : Le menu interactif, et réutilisation des méthodes créées :

Lorsqu'un utilisateur est connecté, au-dessus du menu est affiché le compte connecté.

```
if utilisateur_connecte is not None:
    print(f"Connecté : {utilisateur_connecte['prenom']} {utilisateur_connecte['nom']} {utilisateur_connecte['region']}.")
else:
    print("Non connecté.")
```

1 - On commence par : Afficher les possibilités à l'utilisateur.



On définit la méthode main_menu():

Il s'agit de la boucle du menu principal.

On le fait avec une boucle while, où principalement on met la méthode afficher_menu(), et on y fait rentrer dans la variable choix = input("Votre choix : ")

Et on lance un choix en lien avec la méthode que l'on veut lancer :

Choix 1 :

```python
25        if choix == "1":
26            print("Connexion en cours...")
27            login = input("Login : ")
28            mot_de_passe = input("Mot de passe : ")
29
30            resultat = authentifier(login, mot_de_passe)
31
32            if resultat is not None:
33                print(f"Connexion réussie !")
34                print(f"Bienvenue {resultat['prenom']} {resultat['nom']}")
35                print(f"Type de compte : {resultat['type']}")
36                print(f"Niveau de droits : {resultat['niveau_droits']}")
37
38            else:
39                print("Echec de connexion !")
40                print("Login ou mot de passe incorrect")
41
42            input("Appuyez sur Entrée pour continuer...........")
```

La connexion agit avec la méthode authentifier, vu plus haut.

Choix 2 :

```python
elif choix == "2":
    print("Création d'un utilisateur en cours...")
    nom = input("Nom : ")
    prenom = input("Prénom : ")

    new_user = User(nom, prenom)
    save_user(new_user, "user")

    print(f"L'utilisateur {new_user.login} a été créé avec succès")
    input("Appuyez sur Entrée pour continuer.......")
```

La création d'utilisateur agit directement avec la classe objet User.

Choix 3 :

```python
elif choix == "3":
    print("Création d'un administrateur en cours...")
    nom = input("Nom : ")
    prenom = input("Prénom : ")

    print("Niveau de droits :")
    print("1 - Administrateur ( Il peut gérer les utilisateurs )")
    print("2 - SuperAdministrateur ( Il peut gérer les utilisateurs e
    niveau = input("Choisissez le niveau ( 1 ou 2 ) : ")

    if niveau == "1":
        niveau_droits = 1
    elif niveau == "2":
        niveau_droits = 2
    else:
        print("Niveau invalide. Droits User appliqués")
        niveau_droits = 0 #Droits par défaut quand il y a eu une erre

    nouvel_admin = Admin(nom, prenom, niveau_droits) # Rappel : Un su

    save_user(nouvel_admin, "admin")
    print(f"Administrateur {nouvel_admin.login} créé avec succès")
    print(f"Niveau de droits : {nouvel_admin.niveau_droits}")
    input("Appuyez sur Entrée pour continuer...........")
```

La création d'administrateur agit directement avec la classe Admin

Choix 4 :

```python
elif choix == "4":
    print("Chargement des utilisateurs...")

    utilisateurs = charger_users()

    if len(utilisateurs) == 0:
        print("Aucun utilisateur enregistré")
    else:
        print(f"Nombre total : {len(utilisateurs)} utilisateurs dans le fichier.")
        print(f"{'type':<12} / {'nom':<15} / {'prenom':<15} / {'login':<20} / {'niveau':<8}") # Permet juste d'aligner correctement vers la gauche de x caractères
        print("-------------------------------------------------------------------------------------------------------------")

        for user in utilisateurs:
            print(f"{user['type']:<12} / {user['nom']:<15} / {user['prenom']:<15} / {user['login']:<20} / {user['niveau_droits']:<8}")

    input("Appuyez sur Entrée pour continuer......")
```

L'affichage des utilisateurs agit directement avec la méthode charger_users. Après, on utilise des astuces pour afficher correctement les informations récupérées.

Modification du code existant :

A ce moment là du projet, après avoir fait un point, et nous avions oublié plusieurs points du projet. Nous avons décidé de rajouter la région :

- Nous comptons modifier le CSV et y ajouter une colonne region
- Modification des classes User et Admin pour ajouter le paramètre region
- Modification de save_user() pour sauvegarder la région

Puis, ajouter le système de connexion obligatoire :

- Création d'une variable user_connected
- Modification du menu pour vérifier la connexion
- Ajouter l'option se déconnecter

Nous ajoutons aussi la limitation de tentatives pour le mot de passe.

- Modification de la fonction d'authentification avec une boucle

Et nous finissons les modifications avec la gestion des droits par région, par compte connecté.

- Filtre des users par région de l'admin
- Bloquer la création de plus qu'un seul administrateur
- Ajout des droits au superadmin afin de tout gérer

Ajout de la région :

Modification du CSV :

```python

def init_fichiers():
    """Initialise le fichier en CSV, et si le fichier n'existe pas, il le créé"""
    if not os.path.exists("data"):
        print("Création du dossier data")
        os.makedirs("data")
    else:
        print("Le dossier existe déjà")

    print(f"Verification de l'existance du fichier {FICHIER_USERS}")
    if not os.path.exists(FICHIER_USERS):
        print("Création du fichier users.csv")
        with open(FICHIER_USERS, 'w', newline='', encoding='utf-8') as fichier:
            writer = csv.writer(fichier)
            writer.writerow(['nom', 'prenom', 'login', 'password', 'type', 'niveau_droits', 'region']) # '' et "" ag
        print(f"Fichier {FICHIER_USERS} créé")
    else:
        print("Le fichier existe déjà")

```

On ajoute l'attribut region dans le dictionnaire.

```python
    def __init__(self, nom, prenom, region):
        print("Création d'un utilisateur...")
        Person.__init__(self, nom, prenom)
        self.login = self.generer_login(nom, prenom)
        self.region = region
```

On ajoute dans la classe User la region dans le constructeur init.

```python
class Admin(User):
    """Classe héritant de User, qui hérite lui meme de Person,

    def __init__(self, nom, prenom, niveau_droits=1):
        print("Création d'un admin...")
        User.__init__(self, nom, prenom, region)
        self.niveau_droits = niveau_droits
```

On ajoute la definition d'un admin par la region aussi.

```python
def save_user(user, type_user):
    """Save le user dans le fichier csv"""

    init_fichiers()

    with open(FICHIER_USERS, 'a', newline='', encoding='utf-8') as fichier: # a c'est pour ajouter, alors que w c'es
        writer = csv.writer(fichier)

        if type_user == "admin":
            niveau = user.niveau_droits
        else:
            niveau = 0

        writer.writerow([user.nom, user.prenom, user.login, user.password, type_user, niveau, user.region])

    print(f"Le compte {type_user.capitalize()} {user.login} est sauvegardé")
```

Dans le fichier csv_manager.py, on ajoute aussi la region dans la partie d'ajout dans la fonction save_user. (Lorsqu'on ajoute un user, la region s'ajoute dans une colonne supplémentaire).

Dans le Menu.py, on ajoute l'entrée du choix de la région dans l'option 2.

```python
    elif choix == "2":
        print("Création d'un utilisateur en cours...")
        nom = input("Nom : ")
        prenom = input("Prénom : ")

        print("Choix parmi les régions suivantes :")
        print("1 - Marseille")
        print("2 - Rennes")
        print("3 - Grenoble")
        choix_region = input("Choisissez la région ( 1, 2 ou 3)")

        if choix_region == "1":
            region = "Marseille"
        elif choix_region == "2":
            region = "Rennes"
        elif choix_region == "3":
            region = "Grenoble"
        else:
            print("Region invalide ! Region par défaut : Marseille.")
            region = "Marseille"

        new_user = User(nom, prenom, region)
        save_user(new_user, "user")

        print(f"L'utilisateur {new_user.login} a été créé avec succès dans la région {region}")
        input("Appuyez sur Entrée pour continuer.......")
```

Modification de la création d’administrateur pour la région, avec pareil que plus haut, un choix de région.

```python
elif choix == "3":
    print("Création d'un administrateur en cours...")
    nom = input("Nom : ")
    prenom = input("Prénom : ")

    print("Choix parmi les régions suivantes :")
    print("1 - Marseille")
    print("2 - Rennes")
    print("3 - Grenoble")
    choix_region = input("Choisissez la région ( 1, 2 ou 3)")
    if choix_region == "1":
        region = "Marseille"
    elif choix_region == "2":
        region = "Rennes"
    elif choix_region == "3":
        region = "Grenoble"
    else:
        print("Region invalide ! Region par défaut : Marseille.")
        region = "Marseille"

    print("Niveau de droits :")
    print("1 - Administrateur ( Il peut gérer les utilisateurs de sa région )")
    print("2 - SuperAdministrateur ( BIGBOSS )")
    niveau = input("Choisissez le niveau ( 1 ou 2 ) : ")

    if niveau == "1":
        niveau_droits = 1
    elif niveau == "2":
        niveau_droits = 2
    else:
        print("Niveau invalide. Droits User appliqués")
        niveau_droits = 0 #Droits par défaut quand il y a eu une erreur

    nouvel_admin = Admin(nom, prenom, region, niveau_droits) # Rappel : Un superadmin est un admin avec + de droits

    save_user(nouvel_admin, "admin")
    print(f"Administrateur {nouvel_admin.login} créé avec succès")
    print(f"Region : {region}")
    print(f"Niveau de droits : {nouvel_admin.niveau_droits}")
    input("Appuyez sur Entrée pour continuer...........")
```

IMPORTANT : ACTIVER L'AUTOSAVE DANS FILE SINON IL FAUDRA FAIRE CTRL + S

Puisqu'à ce moment de notre projet, nous avions déjà la connexion mais pas comme demandé, nous avons décidé d'ajouter une variable afin d'être sûr que l'admin/superadmin puisse être les seuls à faire les changements nécessaires.

La connexion :

On détermine au début la variable global utilisateur_connecte en None, parce que personne n'est connecté au départ.

La global permet d'avoir le même statut de la variable partout entre les fonctions, et qu'il ne se remette pas en None. La valeur de utilisateur_connecte doit persister.

```python
from classes.User import User
from classes.Admin import Admin
from utils.csv_manager import save_user, charger_users
from utils.Auth import authentifier

utilisateur_connecte = None

def afficher_menu():
    print("-------------------------")
    print("SYSTEME DE GESTION DES UTILISATEURS")

    if utilisateur_connecte is not None:
        print(f"Connecté : {utilisateur_connecte['prenom']} {utilisateur_connecte['nom']} {utilisateur_connecte['region']}.")
    else:
        print("Non connecté.")
```

Ici, on a la connexion, donc si l'utilisateur n'est pas None, alors on a la connexion.

```python
def verification_connexion():
    """On verifie si un admin est co"""
    if utilisateur_connecte is None:
        print("Accès refusé !")
        print("Vous devez être connecté avec un compte administrateur pour accéder à cette interface.")
        input("Appuyez sur Entrée pour continuer........")
        return False
    return True
```

La fonction permet ici de vérifier si un admin est connecté.

Si l'utilisateur connecté est vide (donc pas connecter, alors accès refusé !)

Et on ajoute à toutes les fonctions nécessaires :

```python
if not verification_connexion():
                break
```

La limitation de droits selon la région :

Affichage :

```python
elif choix == "4":
    if not verification_connexion():
        break

    print("Chargement des utilisateurs...")

    utilisateurs = charger_users()

    if utilisateur_connecte['niveau_droits'] == "1":
        utilisateurs_filtres = []

        for user_filtre in utilisateurs:
            if user_filtre['region'] == utilisateur_connecte['region']:
                utilisateurs_filtres.append(user_filtre)
```

Ici, on remplit la table utilisateur_filtres des utilisateurs qui ont la même région que l'utilisateur connecté (append = ajouter).

Création d'utilisateur, determination de la région selon la région de l'admin :

```python
elif choix == "2":
    if not verification_connexion():
        break

    print("Création d'un utilisateur en cours...")
    nom = input("Nom : ")
    prenom = input("Prénom : ")

    if utilisateur_connecte['niveau_droits'] == "1":
        region = utilisateur_connecte['region']
        print(f"Région : {region} ( votre région )")
    else:
        print("Choix parmi les régions suivantes :")
        print("1 - Marseille")
        print("2 - Rennes")
        print("3 - Grenoble")
        choix_region = input("Choisissez la région ( 1, 2 ou 3)")

        if choix_region == "1":
            region = "Marseille"
        elif choix_region == "2":
            region = "Rennes"
        elif choix_region == "3":
            region = "Grenoble"
        else:
            print("Region invalide ! Region par défaut : Marseille.")
            region = "Marseille"
```

Si l'utilisateur connecté a le niveau de droits 1, qui correspond à 1, alors il va determiner la région de l'utilisateur créer par sa propre région.

Sinon, il choisit (c'est-à-dire s'il est superadmin)

Filtre de l'affichage par région :

```python
    input("Appuyez sur Entrée pour continuer..... ")

elif choix == "4":
    if not verification_connexion():
        break

    print("Chargement des utilisateurs...")

    utilisateurs = charger_users()

    if utilisateur_connecte['niveau_droits'] == "1":
        utilisateurs_filtres = []
        for user_filtre in utilisateurs:
            if user_filtre['region'] == utilisateur_connecte['region']:
                utilisateurs_filtres.append(user_filtre)
                print(f"Région : {utilisateur_connecte['region']}")
    else:
        utilisateurs_filtres = utilisateurs
        print("Toutes les régions pour vous Monsieur/Madame le superadmin BIGBOSS")

    if len(utilisateurs_filtres) == 0:
        print("Aucun utilisateur enregistré")
    else:
        print(f"Nombre total : {len(utilisateurs_filtres)} utilisateurs dans le fichier.")
        print(f"{'type':<12} / {'nom':<15} / {'prenom':<15} / {'login':<20} / {'niveau':<8} / {'region':<10}") # Permet juste d'aligner correctement vers la gauche
        print("-------------------------------------------------------------------------------------------------------------")

        for user in utilisateurs_filtres:
            print(f"{user['type']:<12} / {user['nom']:<15} / {user['prenom']:<15} / {user['login']:<20} / {user['niveau_droits']:<8} / {user['region']:<10}")

    input("Appuyez sur Entrée pour continuer.....")
```

On affiche uniquement les éléments de utilisateurs_filtres.

utilisateurs_filtres se determine selon la region de l'admin connecté.

Si l'admin connecté est un superadmin (donc différent de 1), alors cela affiche tous les utilisateurs du fichier CSV, parcer que utilisateurs_filtres devient utilisateurs.

Recherche par login :

```python
        input("Appuyez sur Entrée pour continuer..... ")
    elif choix == "5":
        if not verification_connexion():
            break
        print("Recherche d'un utilisateur...")

        login_recherche = input("Entrez le login à chercher : ")

        utilisateurs = charger_users()

        if utilisateur_connecte['niveau_droits'] == "1":
            utilisateurs_filtres = []
            for user_filtre in utilisateurs:
                if user_filtre['region'] == utilisateur_connecte['region']:
                    utilisateurs_filtres.append(user_filtre)
        else:
            utilisateurs_filtres = utilisateurs

        utilisateur_trouve = None
        for user_filtre in utilisateurs_filtres:
            if user_filtre['login'] == login_recherche:
                utilisateur_trouve = user_filtre
                break

        if utilisateur_trouve is not None:
            print("Utilisateur trouvé !")
            print(f"Nom : {utilisateur_trouve['nom']}")
            print(f"Prénom : {utilisateur_trouve['prenom']}")
            print(f"Login : {utilisateur_trouve['login']}")
            print(f"Type : {utilisateur_trouve['type']}")
            print(f"Niveau de droits : {utilisateur_trouve['niveau_droits']}")
            print(f"Région : {utilisateur_trouve['region']}")
        else:
            print(f"L'utilisateur {login_recherche} est introuvable")
            if utilisateur_connecte['niveau_droits'] == "1":
                print(f"Recherche limité à la région {utilisateur_connecte['region']}")

        input("Appuyez sur Entrée pour continuer....")
```

Au début, on demande à l'utilisateur d'entrer le login à rechercher.

On charge tous les users.

Si l'utilisateur connecté est un admin, alors on créé la table utilisateurs_filtres, et pour chaque user_filtre dans les utilisateurs, si la région est la même que l'utilisateur connecté, alors on ajoute ce user à la table utilisateurs_filtres. Sinon, la table utilisateurs_filtres corresponds à tous les users (Cas pour le superadmin)

On initialise à None la variable utilisateur_trouve.

Et on refait pareil, cette fois en comparant la valeur dans une boucle de chaque éléments de la table utilisateurs_filtres.

Si utilisateur trouve n'est pas vide (donc en soit, on l'a trouvé), alors on affiche les informations, sinon il est introuvable, et on rappelle que la recherche est limitée à la région de l'utilisateur connecté.

Modification du fichier CSV :

Seul un administrateur / superadmin peut modifier les données du fichier CSV.

Yen a des boucles dans ce choix.

```
elif choix == "6":
    if not verification_connexion():
        break
    print("Modification en cours...")
```

On commence par la vérification de connexion.

Ensuite, on fait un input pour demander le login à modifier, et on stocke cette information dans login_modification.

On charge dans la variable utilisateurs la liste des users du fichier CSV.

On définit une variable utilisateur_trouve à None pour être sûr qu'il n'y ait rien de base dedans, avant de le remplir par la réponse que l'on souhaite. (l'utilisateur avec la bonne valeur login pour le modifier.)

Ensuite, on définit un index utilisateur, qui va nous permettre de garder à quelle position est l'utilisateur. On l'initialise à -213, comme ça si on ne trouve pas, cela sera différent d'une position connue par le fichier, et on retournera un nope.

```
for compteur, user_filtre in enumerate(utilisateurs):
    if user_filtre['login'] == login_modification:
        if utilisateur_connecte['niveau_droits'] == "1":
            if user_filtre['region'] != utilisateur_connecte['region']:
                print(f"Accès refusé ! L'utilisateur est dans la region {user_filtre['region']}, mauvaise région ! Sinon connectez vous en SUPERADMIN !")
                input("Appuyez sur Entrée pour continuer.....")
                break
            if user_filtre['type'] == 'admin':
                print("Accès refusé ! Connectez vous en SUPERADMIN pour modifier un admin !")
                input("Appuyez sur Entrée pour continuer....")
                break
        utilisateur_trouve = user_filtre
        index_user = compteur
        break
```

Ensuite, on lance un compteur (c'est ça qui va nous servir à remplacer la valeur index_user si on trouve la position du user que l'on recherche). On recherche alors la position.

Et on lance aussi user_filtre qui va nous permettre de trouver la valeur que l'on recherche.

En clair, on cherche la valeur, et le nombre de boucle nous permettra de trouver la position (la ligne) où il y a ce que l'on recherche.

Si la valeur login dans la première boucle corresponds à la valeur login input par l'utilisateur, ET :

Si l'utilisateur ayant demandé ça est connecté sur un compte administrateur, ET :

Si l'utilisateur trouvé n'est pas dans la même région que le compte connecté, ALORS :

On affiche un accès refusé, MAIS :

Si l'utilisateur trouvé a un type == 'admin' , alors on refuse l'accès aussi.

Si l'utilisateur connecté est un administrateur, et qu'il est dans la même région que l'utilisateur recherché, et que l'utilisateur recherché n'est pas un administrateur, alors on met l'utilisateur recherché qui est désormais trouvé dans utilisateur_trouve, et on garde sa position dans index_user.

```
if utilisateur_trouve is not None:
    print(f"Utilisateur trouvé : {utilisateur_trouve['prenom']}, {utilisateur_trouve['nom']}")
    print("Que voulez-vous modifier ? 1 = Nom, 2 = Prénom, 3 = Région et 4 = Annuler")

    choix_modification = input("Votre choix : ")

    if choix_modification == "1":
        nouveau_nom = input("Nouveau nom : ")
        utilisateurs[index_user]['nom'] = nouveau_nom
        print(f"Nom modifié : {nouveau_nom}")
    elif choix_modification == "2":
        nouveau_prenom = input("Nouveau prénom : ")
        utilisateurs[index_user]['prenom'] = nouveau_prenom
        print(f"Prénom modifié : {nouveau_prenom}")
    elif choix_modification == "3":
        if utilisateur_connecte['niveau_droits'] == "1":
            print("Seul un SUPERADMIN peut toucher à ça !")
        else:
            print("Région disponible : 1 - Marseille / 2 - Rennes / 3 - Grenoble")
            choix_region = input("1, 2 ou 3 : ")

            if choix_region =="1":
                utilisateurs[index_user]['region'] = "Marseille"
            elif choix_region == "2":
                utilisateurs[index_user]['region'] = "Rennes"
            elif choix_region == "3":
                utilisateurs[index_user]['region'] = "Grenoble"
            else:
                print("Région indisponible !")
                input("Appuyez sur Entrée pour continuer......")
                break

            print(f"Région modifiée : {utilisateurs[index_user]['region']}")
```

Donc à partir de là, on a l'utilisateur recherché qui est bien dans la variable utilisateur_trouve, et on a sa position dans le fichier csv.

Si l'utilisateur trouvé a bien passé toutes les vérifications précédentes, alors :

On affiche ses informations, et on demande ce que l'on veut modifier dessus.

On fait la modification en créeant une variable choix_modification qui va nous permettre de toucher à la valeur que l'utilisateur connecté veut modifier.

En suite on définit pour changer le nom et le prenom 2 variables, où on remplacera dans : utilisateurs, à la position indiquée par index_user, le nom ou le prénom, selon le choix entré.

Pour la modification de la région, seul un SUPERADMIN peut le faire. Alors on applique une verification du niveau de droits pour voir si l'utilisateur connecté est bien un SUPERADMIN, soit niveau_droits == "2".
S'il le peut, alors c'est bon il change la variable region de l'utilisateur indexé dans utilisateurs, la BDD.

```
    elif choix_modification == "4":
        print("Modification annulée !")
        input("Appuyez sur Entrée pour continuer ......")
        break
```

On laisse quand même une porte de sortie si l'utilisateur connecté veut annuler la modification.

Si le choix de modification a été le 1, 2 ou 3, alors on supprime le fichier users.

RAPPEL ! Le supprimer n'est pas grave, puisque l'on garde les informations du fichier dans utilisateur, qui a utilisé la fonction charger_users() (information ici garder dans la RAM du PC), avant de la réécrire.

On initialise le fichier, et on ouvre le fichier : FICHIERS_USERS (correspond au chemin), en mode 'a', appends = ajouter, avec chaque nouvelle ligne initialisée, en UTF-8 pour le format français.

Et on réécrit le fichier avec les informations stockées dans utilisateurs grâce à charger_users().

```python
if choix_modification in ["1", "2", "3"]:
    os.remove(FICHIER_USERS)

    init_fichiers()

    with open(FICHIER_USERS, 'a', newline='', encoding='utf-8') as fichier:
        writer = csv.writer(fichier)
        for user in utilisateurs:
            writer.writerow([user['nom'],user['prenom'],user['login'],user['password'],user['type'],user['niveau_droits'], user['region']])

    print("Modifications sauvegardées !")

elif utilisateur_trouve is None and index_user == -213:
    print("Utilisateur introuvable !")

input("Appuyez sur Entrée pour continuer.....")
```

Et si utilisateur_trouve est None, et l'index == -213, alors on affiche que l'utilisateur est introuvable.

PS : Je viens de me rendre compte que le and ne sert à rien, c'est une vérification inutile. Mais jle laisse

Suppression d'un utilisateur :

```python
login_suppression = input("Entrez le login de l'utilisateur à supprimer : ")

utilisateurs = changer_users()

utilisateur_trouve = None
index_trouve = -213

for compteur, user_filtre in enumerate(utilisateurs):
    if user_filtre['login'] == login_suppression:
        if utilisateur_connecte['niveau_droits'] == "1":
            if user_filtre['region'] != utilisateur_connecte['region']:
                print(f"Accès refusé ! L'utilisateur est dans la region {user_filtre['region']}, mauvaise région ! Sinon connectez vous en SUPERADMIN !")
                input("Appuyez sur Entrée pour continuer.....")
                break
            if user_filtre['type'] == 'admin':
                print("Accès refusé ! Connectez vous en SUPERADMIN pour supprimer un admin !")
                input("Appuyez sur Entrée pour continuer....")
                break
        utilisateur_trouve = user_filtre
        index_user = compteur
```

Même façon pour la recherche de l'utilisateur afin de le modifier, avec la fonction enumerate(BDD).

Tout pareil que la modification, sauf que l'on va supprimer l'utilisateur avec la méthode pop.

PROJET T1 FINI

Links :

https://docs.python.org/fr/3/tutorial/

https://docs.python.org/fr/3/library/csv.html (DICTREADER + exemples)

https://docs.python.org/fr/3/library/hashlib.html (SHA 256)

https://docs.python.org/fr/3/library/os.html (7.2.1)

https://docs.python.org/fr/3/tutorial/inputoutput.html#reading-and-writing-files (Gestion de fichiers OS)

Et les cours de Mme. OULMI

Après soutenance :

Mise en place de certains fix :

Après la soutenance, certains soucis étaient évidents, et on les a fix :

```
EXPLORER
OPEN EDITORS
PROJETPYTHONT1
  __pycache__
  classes
    __pycache__
    Admin.py
    Person.py
    User.py
  data
    users.csv
  utils
    __pycache__
    Auth.py
    csv_manager.py
    password_utils.py
  .gitignore
  main.py
  Menu.py
```

```
enu.py   .gitignore   password_utils.py   User.py   Person.py   csv_manager.py   Admi
classes > User.py > User > __init__
You, 1 hour ago | 2 authors (You and one other)
from classes.Person import Person
from utils.password_utils import generer_mdp, hash_mdp
from utils.csv_manager import charger_users

You, 1 hour ago | 2 authors (You and one other)
class User(Person):
    """Classe qui hérite de Person, et donc qui récupère nom et prenom ainsi que les méthodes dec

    def __init__(self, nom, prenom, region):
        print("Création d'un utilisateur...")
        Person.__init__(self, nom, prenom)
        self.login = None # Pas de méthode dans un constructeur      You, 1 hour ago • Fix tenta
        self.region = region

        mdp_clair = generer_mdp()
        print(f"Mot de passe généré : {mdp_clair}")
        self.password = hash_mdp(mdp_clair)

    def generer_login(self, nom, prenom):
        """Generation du login via le nom et prenom"""
        premiere_lettre = prenom[0]
        login_base = premiere_lettre + nom
```

Dans le classe objet User qui hérite de Person, nous avions mis une méthode dedans. Désormais, la méthode est faite en dehors, et self.login est initialisé à None, c'est donc vide.

```
Menu.py C:\Users\malop\Desktop\Cours Python 3eme année\ProjetPythonT1 - References (6)
def main_menu():
                region = "Rennes"
            elif choix_region == "3":
                region = "Grenoble"
            else:
                print("Region invalide ! Region par défaut : Marseille.")
                region = "Marseille"

        new_user = User(nom, prenom, region)
        new_user.generer_login(nom, prenom)
        save_user(new_user, "user")

        print(f"L'utilisateur {new_user.login} a été créé avec succès dans la région {re
        input("Appuyez sur Entrée pour continuer.......")

Menu.py                                  2
  User import User
  new_user = User(nom, prenom, region)
Admin.py classes                         3
User.py classes                          1
  class User(Person):
```

Et la méthode de génération est appelée une seule fois, ici dans Menu.py , pour générer le login lors de la création d'un utilisateur.

Tentatives maximum de connexion :

De plus, nous avons rajouter une boucle qui compte le nombre de tentative et qui bloque les tentatives de connexion au-dessus de 3.

```python
def main_menu():
        if choix == "1":
            global utilisateur_connecte

            print("Connexion en cours...")
            login = input("Login : ")

            tentatives = 0
            max_tentative = 3

            while tentatives < max_tentative:
                mot_de_passe = input(f"{tentatives + 1} Attention, le maximum est de {max_tentative}. Mot de passe : ")

                resultat = authentifier(login, mot_de_passe)

                if resultat == -2:
                    print("Login introuvable")
                    break

                elif resultat == -1: # Lorsque le return de authentifier() est à -1, donc lorsque la connexion a échouée
                    tentatives += 1
                    if tentatives < max_tentative:
                        print(f"Il vous reste {max_tentative - tentatives} tentatives !")
                    else:
                        print("Nombre maximum de tentatives atteint, retour automatique au menu !")
                else:
                    if resultat['type'] == 'admin':
                        utilisateur_connecte = resultat
                        print("Connexion réussie !")
                        print(f"Bienvenue {resultat['prenom']} {resultat['nom']}.")
                        print(f"Vous êtes affilié à la région {resultat['region']}.")
                        print(f"Votre niveau de droits : {resultat['niveau_droits']} {resultat['type']}.")
                        break
                    else:
                        print("Connexion refusée.")
                        print("Seuls les administrateurs peuvent se connecter.")
                        break

                input("Appuyez sur Entrée pour continuer.....")
```

Vérification de connexion :

Désormais, la vérification de connexion se finit par un continue plutôt qu'un break (trop agressif)

```python
elif choix == "2":
    if not verification_connexion():
        continue
```

Projet T2 : Gestion de fichiers

Pas de / au début !

1 - Création de la structure / Organisation :

Comme il faut des dossiers pour chaque région, nous allons utiliser fichiers_marseille, fichiers_rennes, fichiers_grenoble.

```
> fichiers_grenoble
> fichiers_marseille
> fichiers_rennes
```

On utilisera les bibliothèques os et shutil

Faisons 2 fonctions :
Une fonction pour initialiser les dossiers régionaux, ce qui va vérifier si le ou les dossiers sont créés, et une autre fonction pour retourner le chemin du dossier selon la région de l'administrateur connecté.

Fonction init_dossiers_regions() :

```
ers.csv ↓M   Admin.py ↓M   Person.py   Menu.py ↓M, M   file_manager.py U ×
utils > file_manager.py > ...
1  import os
2  import shutil
3
4  def init_dossiers_regions():
5      """Initialise les dossiers des régions si ça n'existe pas"""
6
7      regions = ["marseille", "rennes", "grenoble"]
8
9      for region in regions:
10         nom_dossier = f"fichiers_{region}"
11         if not os.path.exists(nom_dossier):
12             os.mkdir(nom_dossier)
13             print(f"Dossier {nom_dossier} créé")
14         else:
15             print(f"Dossier {nom_dossier} existe déjà.")
```

Pour éviter de répéter 3 fois le même code, on utilise une liste contenant les 3 régions.

La boucle for pour que chaque region dans la liste prendra les valeurs marseille, rennes puis grenoble.

```python
for region in regions:
    nom_dossier = f"fichiers_{region}"
    if not os.path.exists(nom_dossier):
        os.mkdir(nom_dossier)
        print(f"Dossier {nom_dossier} créé")
    else:
        print(f"Dossier {nom_dossier} existe déjà.")
```

os.mkdir créer le dossier

```python
os.mkdir(nom_dossier)
```

Le if not os.path.exists() permet d'éviter une erreur si le dossier existe déjà.

```python
if not os.path.exists(nom_dossier):
```

Fonction obtenir_chemin_region(region) :

```python
def obtenir_chemin_region(region):
    """Retourne le chemin du dossier correspondant à la région donnée"""
    region_lower = region.lower()

    chemin = f"fichiers_{region_lower}/"

    return chemin
```

On met en minuscule pour éviter les soucis si le user met en majuscule etc...

On ajoute cela dans le Menu.py :

```python
from utils.file_manager import init_dossiers_regions, obtenir_chemin_region
```

Et on ajoute la fonction dans le main_menu()

```python
def afficher_menu():
    else:
        print("Non connecté.")

    print("1 - Se connecter")
    print("2 - Créer un utilisateur")
    print("3 - Créer un administrateur")
    print("4 - Afficher tous les utilisateurs")
    print("5 - Rechercher un utilisateur")
    print("6 - Modifier un utilisateur")
    print("7 - Supprimer un utilisateur")
    print("8 - Se déconnecter")
    print("9 - Quitter (proprement sans ctrl+C)")
    print("-----------------------------------------------------------------")

def main_menu():
    """Boucle du menu principal"""
    print("Initialisation des dossiers des régions...")
    init_dossiers_regions()
    print(".........")

    while True:
        afficher_menu()
        choix = input("Votre choix : ")

        if choix == "1":
            global utilisateur_connecte

            print("Connexion en cours...")
            login = input("Login : ")
```

La fonction naviguer_arbo() :

Pour créer l'effet visuel d'arborescence, multiplions le niveau d'arborescence par une indentation.

```python
def naviguer_arbo(chemin, niveau=0):
    """Afficher les dossiers et fichiers appartenant à une région"""

    if not os.path.exists(chemin):
        print(f"Erreur : Le chemin {chemin} n'existe pas.")
        return

    indentation = "  " * niveau

    try:
        elements = os.listdir(chemin)

        if len(elements) == 0:
            print(f"{indentation}(Dossier vide)")
            return

        for element in elements:
            chemin_complet = os.path.join(chemin, element)

            if os.path.isdir(chemin_complet):
                print(f"{indentation}Dossier :{element}/")
                naviguer_arbo(chemin_complet, niveau + 1)
            else:
                print(f"{indentation}Fichier :{element}")

    except PermissionError:
        print(f"{indentation} Accès refusé !")
```

Si la taille de elements est à 0, alors le dossier est vide.

Pour chaque element dans elements, on récupère le chemin et le l'element qui vont former le chemin_complet. Et via ce chemin_complet, on y determine une indentation selon le niveau de chemin. Le niveau tout en haut est un dossier, donc on met un Dossier, et le niveau en-dessous sera toujours un fichier dons on le met en Fichier.

Et si la permission n'est pas bonne, alors on met accès refusé.

```
EXPLORER
PROJETPYTHONT1
  __pycache__
  classes
    __pycache__
    Admin.py
    Person.py
    User.py
  data
    users.csv
  fichiers_grenoble
  fichiers_marseille
  fichiers_rennes
  utils
    __pycache__
    Auth.py
    csv_manager.py
    file_manager.py M
    password_utils.py
  main.py
  Menu.py M

main.py | User.py | users.csv | Admin.py | Person.py | Menu.py M | file_manager.py M
Menu.py > main_menu
41  def main_menu():
372         elif choix == "10":
373             if not verification_connexion():
374                 continue
375
376             print("Navigation dans l'arborescence...")
377
378             if utilisateur_connecte['niveau_droits'] == "1":
379                 chemin = obtenir_chemin_region(utilisateur_connecte['region'])
380                 print(f"Arborescence de la région ({utilisateur_connecte['region']}) :")
381                 print()
382                 naviguer_arbo(chemin)
383             else:
384                 print("Choisissez la région à explorer :")
385                 print("1 - Marseille")
386                 print("2 - Rennes")
387                 print("3 - Grenoble")
388                 choix_region = input("Votre choix (1, 2 ou 3) : ")
389
390                 if choix_region == "1":
391                     region = "Marseille"
392                 elif choix_region == "2":
393                     region = "Rennes"
394                 elif choix_region == "3":
395                     region = "Grenoble"
396                 else:
397                     print("Choix invalide !")
398                     input("Appuyez sur Entrée pour continuer...")
399                     continue
400
401                 chemin = obtenir_chemin_region(region)
402                 print(f"Arborescence de la région {region} :")
403                 naviguer_arbo(chemin)
404
405
406         else:
407             print("Veillez à entrer un chiffre entre 1 et 10 inclus !")
```

2 - Créer des répertoires :

On établit ce que peut faire un administrateur avec la création de répertoire. Il pourra créer un dossier ou un fichier.

```python
def naviguer_arbo(chemin, niveau=0):
        print(f"{indentation} Accès refusé !")

def creer_element(chemin_base):
    """Créer un dossier ou un fichier dans le chemin spécifié"""

    print("Créer un fichier ( 1 ) ou un dossier ( 2 ) ?")
    choix = input("Votre choix ( 1 ou 2 ) : ")

    if choix == "1":
        nom_dossier = input("Nom du dossier : ")

        chemin_complet = os.path.join(chemin_base, nom_dossier)

        if os.path.exists(chemin_complet):
            print(f"Erreur : Le dossier '{nom_dossier} existe déjà!")
            return

        os.mkdir(chemin_complet)
        print(f"Dissier créé ! {chemin_complet}")
    elif choix == "2":
        nom_fichier = input("Nom du fichier")

        chemin_complet = os.path.join(chemin_base, nom_fichier)

        if os.path.exists(chemin_complet):
            print(f"Erreur, le fichier '{nom_fichier} existe déjà !")
            return

        print("Contenu du fichier (Entrée pour un fichier vide) : ")
        contenu = input()

        with open(chemin_complet, 'w', encoding='utf-8') as fichier:
            fichier.write(contenu)

            print(f"Fichier créé ! {chemin_complet}")

    else:
        print("Choix invalide !")
```

le with open plutot que open permet de fermer automatiquement l'element après.

On met ça dans le main_menu, et c'est parti :

```python
def main_menu():
                    chemin = obtenir_chemin_region(region)
                    print(f"Arborescence de la région {region} :")
                    naviguer_arbo(chemin)

            elif choix == "11":
                if not verification_connexion():
                    continue
                print("Création d'un répertoire ou d'un fichier...")

                if utilisateur_connecte['niveau_droits'] == "1":
                    chemin = obtenir_chemin_region(utilisateur_connecte['region'])
                    print(f"Création dans votre région ({utilisateur_connecte['region'
                    creer_element(chemin)
                else:
                    print("Dans quelle région voulez-vous créer ?")
                    print("1 - Marseille")
                    print("2 - Rennes")
                    print("3 - Grenoble")
                    choix_region = input("Votre choix 1, 2 ou 3 : ")

                    if choix_region == "1":
                        region = "Marseille"
                    elif choix_region == "2":
                        region = "Rennes"
                    elif choix_region == "3":
                        region = "Grenoble"
                    else:
                        print("Choix invalide !")
                        input("Appuyez sur Entrée pour continuer...")
                        continue

                    chemin = obtenir_chemin_region(region)
                    print(f"Création dans la région de {region}.")
                    creer_element(chemin)

                input("Appuyez sur Entrée pour continuer...")

            else:
                print("Veillez à entrer un chiffre entre 1 et 10 inclus !")
```

En arrivant ici, on a trouvé un soucis évident : On ne peut pas créer de fichier dans un dossier créer. Il ne s'agit de la gestion que au niveau de la racine. On a donc modifier pour permettre à l'utilisateur de rentrer le chemin qu'il veut remplir.

On a donc ajouter la possibilité de mettre le chemin directement.

```python
def creer_element(chemin_base):
    """Créer un dossier ou un fichier dans le chemin spécifié"""

    print("Créer un dossier ( 1 ) ou un fichier ( 2 ) ?")
    choix = input("Votre choix ( 1 ou 2 ) : ")

    if choix == "1":
        print("Vous pouvez utiliser des chemins")
        nom_dossier = input("Nom du dossier ou du chemin : ")

        chemin_complet = os.path.join(chemin_base, nom_dossier)

        if os.path.exists(chemin_complet):
            print(f"Erreur : Le dossier '{nom_dossier} existe déjà!")
            return

        os.makedirs(chemin_complet)
        print(f"Dissier créé ! {chemin_complet}")
    elif choix == "2":
        print("Vous pouvez utiliser des chemins")
        nom_fichier = input("Nom du fichier avec ou sans chemin : ")

        chemin_complet = os.path.join(chemin_base, nom_fichier)

        if os.path.exists(chemin_complet):
            print(f"Erreur, le fichier '{nom_fichier} existe déjà !")
            return

        dossier_parent = os.path.dirname(chemin_complet)
        if not os.path.exists(dossier_parent):
            print(f"Erreur : Le dossier '{dossier_parent}' n'existe pas !")
            print("Il faut créer le dossier d'abord, ou utiliser un chemin existant.")
            return
```

os.path.dirname récupère le chemin du dossier parent

3 - Modifier les fichiers :

La fonction modifier_fichier()

```python
def modifier_fichier(chemin_base):
    """Modifie le contenu d'un fichier existant"""

    print("Nom du fichier à modifier ( avec le chemin si besoin )")
    nom_fichier = input()

    chemin_complet = os.path.join(chemin_base, nom_fichier)

    if not os.path.exists(chemin_complet):
        print(f"Erreur : Le fichier '{nom_fichier} n'existe pas !")
        return

    if not os.path.isfile(chemin_complet):
        print(f"Erreur : '{nom_fichier} est un dossier, pas un fichier !")
        return

    print("\nContenu du fichier : ")
    try:
        with open(chemin_complet, 'r', encoding='utf-8') as fichier:
            contenu_actuel = fichier.read()
            if contenu_actuel:
                print(contenu_actuel)
            else:
                print("(Fichier vide)")

    except Exception as e:
        print(f"Erreur lors de la lecture : {e}")
        return
    print("Fin du contenu\n")

    print("Comment voulez-vous modifier le fichier ? 1 pour remplacer le contenu et 2 pour ajouter du conter
    choix = input("Votre choix, 1 ou 2 : ")
```

Avant le choix de tout supprimer pour modifier (donc remplacer tout le contenu ou ajouter après un saut de ligne du contenu à un fichier), on doit vérifier si ce que l'utilisateur pointe existe, puis s'il est bel et bien un fichier :

```python
if not os.path.exists(chemin_complet):
    print(f"Erreur : Le fichier '{nom_fichier} n'existe pas !")
    return

if not os.path.isfile(chemin_complet):
    print(f"Erreur : '{nom_fichier} est un dossier, pas un fichier !")
    return
```

Ensuite, on affiche le contenu actuel :

```python
print("\nContenu du fichier : ")
try:
    with open(chemin_complet, 'r', encoding='utf-8') as fichier:
        contenu_actuel = fichier.read()
        if contenu_actuel:
            print(contenu_actuel)
        else:
            print("(Fichier vide)")
```

Gestion d'erreur avec except Exeption as e

```python
except Exception as e:
    print(f"Erreur lors de la lecture : {e}")
    return
print("Fin du contenu\n")
```

Puis on demande quel mode etc...

```python
print("Comment voulez-vous modifier le fichier ? 1 pour remplacer le contenu et 2 pour ajouter du conter
choix = input("Votre choix, 1 ou 2 : ")

if choix == "1":
    print("Nouveau contenu du fichier :")
    nouveau_contenu = input()

    with open(chemin_complet, 'w', encoding='utf-8') as fichier:
        fichier.write(nouveau_contenu)

        print(f"Fichier modifié ( tout le contenu a été remplacé !) : {chemin_complet}")

elif choix == "2":
    print("Contenu à ajouter à la fin du fichier :")
    contenu_ajouter = input()

    with open(chemin_complet, 'a', encoding='utf-8') as fichier:
        fichier.write("\n") # Saut de ligne sinon pas beau
        fichier.write(contenu_ajouter)

    print(f"Fichier modifié (Contenu ajouté !): {chemin_complet}")

else:
    print("Choix invalide on t'a dis !")
```

Le mode ‘w’ (donc write, écriture) écrase tout le contenu existant, alors que le mode ‘a’ (donc append, ajouter) conserve le contenu existant.

On ajoute cette fonction dans le dossier Menu.py dans le menu_main() avec toutes les vérifications nécessaires.

```
elif choix == "12":
    if not verification_connexion():
        continue
    print("Modification d'un fichier...")

    if utilisateur_connecte['niveau_droits'] == "1":
        chemin = obtenir_chemin_region(utilisateur_connecte['region'])
        print(f"Modification dans votre région ({utilisateur_connecte['region']}).")
        modifier_fichier(chemin)
    else:
        print("Dans quelle région voulez-vous créer ?")
        print("1 - Marseille")
        print("2 - Rennes")
        print("3 - Grenoble")
        choix_region = input("Votre choix 1, 2 ou 3 : ")

        if choix_region == "1":
            region = "Marseille"
        elif choix_region == "2":
            region = "Rennes"
        elif choix_region == "3":
            region = "Grenoble"
        else:
            print("Choix invalide !")
            input("Appuyez sur Entrée pour continuer...")
            continue

        chemin = obtenir_chemin_region(region)
        print(f"Modification dans la région de {region}.")
        modifier_fichier(chemin)

    input("Appuyez sur Entrée pour continuer...")
```

Copier des répertoires, des fichiers

copier_element():

Lorsque l’on copie un fichier, on copie le fichier.

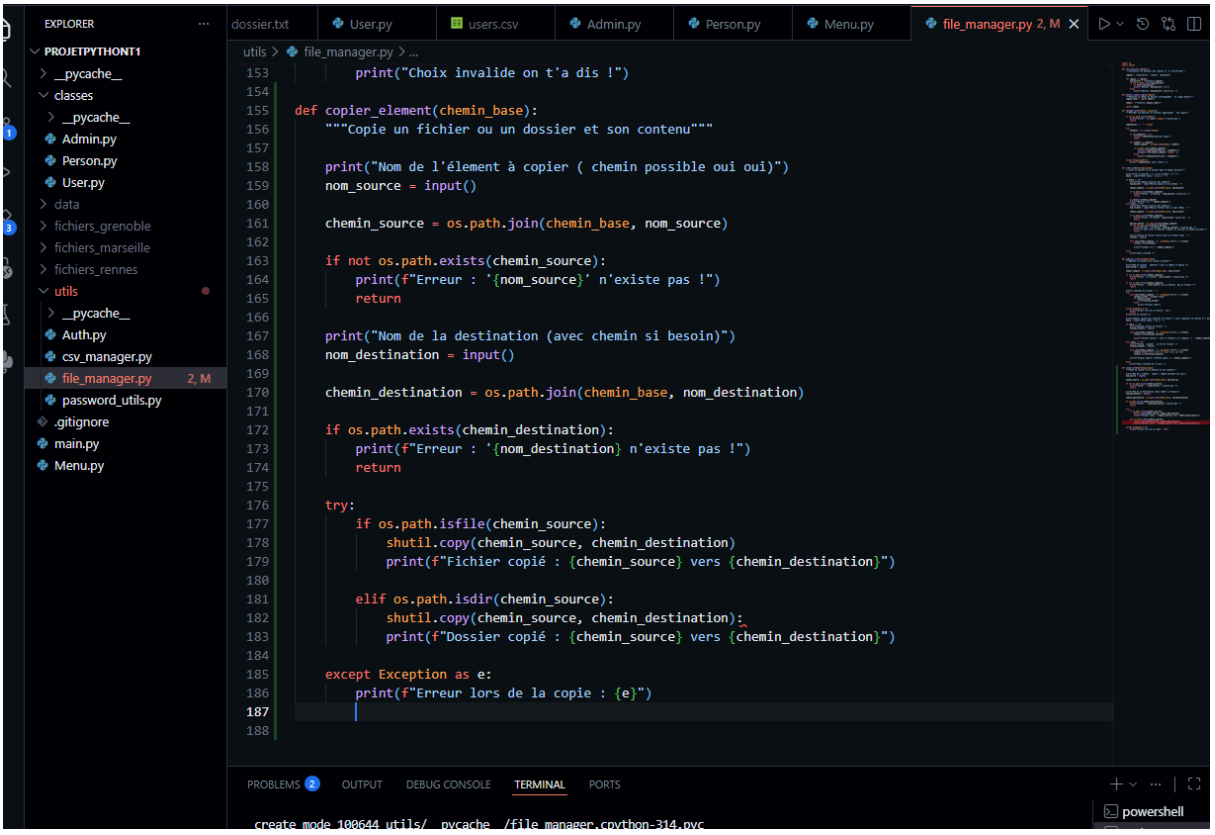
Lorsque l’on copie un dossier, on copie le dossier et son contenu.

On utilisera alors shutil.copy(source, destination) pour copier un fichier

On utilisera aussi shutil, mais le copytree(source, destination) pour un dossier et son contenu.

chemin_base est le chemin du dossier de base.

os.path.join permet que le code fonctionne sur tous les systèmes. (par exemple, les / necessaires sur Windows et les \ qui sont necessaires sur Linux ou Mac. Il fait des + et concatène de ce que j’ai compris.



Déplacer des répertoires, des fichiers

La fonction deplacer_element()

```python
def deplacer_element(chemin_base):
    """Déplacer un fichier ou un dossier"""

    print("Nom de l'élément à déplacer : ")
    nom_source = input()

    chemin_source = os.path.join(chemin_base, nom_source)

    if not os.path.exists(chemin_source):
        print(f"Erreur : '{nom_source}' n'existe pas !")
        return

    print("Nom de la destination : ")
    nom_destination = input()

    chemin_destination = os.path.join(chemin_base, nom_destination)

    if os.path.exists(chemin_destination):
        print(f"Erreur '{nom_destination}' existe déjà !")
        return

    dossier_parent = os.path.dirname(chemin_destination)
    if dossier_parent and not os.path.exists(dossier_parent):
        print(f"Erreur : Le dossier parent '{dossier_parent}' n'existe pas")
        print("Créer d'abord le dossier de destination")
        return

    try:
        shutil.move(chemin_source, chemin_destination)
        print(f"Element deplacé !")

    except Exception as e:
        print(f"Erreur lors du déplacement : {e}")
```

shutil.move() permet de déplacer, et on a pas besoin de vérifier si c'est un fichier ou pas puisqu'il gère les 2.

On le met dans le Menu.py, et on fait les mêmes vérifications.

Supprimer un répertoire, des fichiers

Pour supprimer, il va falloir demander la confirmation à l'utilisateur, et également utiliser 2 fonctions différentes :

Une première fonction pour supprimer un fichier, une deuxième fonction pour supprimer un dossier en récursivité puisqu'on supprimera tout son contenu.

```python
if os.path.isfile(chemin_complet):
    print(f"Fichier à supprimer : {chemin_complet}")
elif os.path.isdir(chemin_complet):
    print(f"Dossier à supprimer : {chemin_complet}")
```

Si ça trouve un fichier dans l'input de l'utilisateur, alors ça montre son chemin.

Si ça trouve un dossier dans l'input de l'utilisateur, alors ça montre son chemin.

Ca permet de montrer le contenu de ce qui va être supprimer par l'utilisateur afin d'être sûr qu'il supprime bien ce qu'il veut.

```python
def supprimer_element(chemin_base):
    """Supprime un fichier ou un dossier"""

    print("ATTENTION, supprimé c'est supprimé reprendre c'est pas possible !")
    print("Nom de l'élément à supprimer : ")
    nom_element = input()

    chemin_complet = os.path.join(chemin_base, nom_element)

    if not os.path.exists(chemin_complet):
        print(f"Erreur : '{nom_element}' n'existe pas !")
        return

    if os.path.isfile(chemin_complet):
        print(f"Fichier à supprimer : {chemin_complet}")
    elif os.path.isdir(chemin_complet):
        print(f"Dossier à supprimer : {chemin_complet}")

    confirmation1 = input("Êtes-vous sûr de supprimer ??? Tapez oui.")
    if confirmation1.lower() != "oui":
        print("Suppression annulée. Il fallait mettre oui. Ou OuI, ou OUI.")
        return

    try:
        if os.path.isfile(chemin_complet):
            os.remove(chemin_complet)
            print(f"Fichier supprimé : {chemin_complet}")
        elif os.path.isdir(chemin_complet):
            shutil.rmtree(chemin_complet)
            print(f"Dossier supprimé : {chemin_complet}")
    except Exception as e:
        print(f"Erreur lors de la suppression : {e}")
```

Partie Serveur FTP (Upload / Download / Versioning)

serveurftp_init():

```python
def serveurftp_init():
    """Initialise le dossier pour le serveur FTP de Paris"""

    serveurftp = "Serveurftp_Paris"

    if not os.path.exists(serveurftp):
        os.mkdir(serveurftp)
        print(f"Dossier du serveur FRP créé avec succès : {serveurftp}")
    else:
        print(f"Dossier {serveurftp} existe déjà soit concentré...")

    regions = ["marseille", "rennes", "grenoble"]

    for region in regions:
        path_region = os.path.join(serveurftp, region)

        if not os.path.exists(path_region):
            os.mkdir(path_region)
            print(f"Dossier de la région FTP bien créé : {path_region}")
        else:
            print(f"Soit concentré... C'est déjà créé ! {path_region}")
```

On initialise et vérifie si le dossier FTP au niveau de Paris et des 3 régions existent bien.

On vient de se rendre compte qu'on avait déjà une fonction pour init des fichiers, donc on la modifie un peu pour y accueillir le fichier FTP Paris.

```python
def init_dossiers_regions():
    """Initialise les dossiers des régions et le serveur FTP si ça n'existe pas"""

    regions = ["marseille", "rennes", "grenoble"]

    for region in regions:
        nom_dossier = f"fichiers_{region}"
        if not os.path.exists(nom_dossier):
            os.mkdir(nom_dossier)
            print(f"Dossier {nom_dossier} créé")
        else:
            print(f"Dossier {nom_dossier} existe déjà.")

    serveurftp = "serveurftp_Paris"

    if not os.path.exists(serveurftp):
        os.mkdir(serveurftp)
        print(f"Dossier du serveur FRP créé avec succès : {serveurftp}")
    else:
        print(f"Dossier {serveurftp} existe déjà soit concentré...")

    regions = ["marseille", "rennes", "grenoble"]

    for region in regions:
        path_region = os.path.join(serveurftp, region)

        if not os.path.exists(path_region):
            os.mkdir(path_region)
            print(f"Dossier de la région FTP bien créé : {path_region}")
        else:
            print(f"Soit concentré... C'est déjà créé ! {path_region}")
```

Donc on rajoute tout simplement ce qu'on avait fait après.

upload_fichier():

```python
def upload_fichier(chemin, region):
    """Upload un fichier avec versionning dans le fichier serveur ftp"""

    print("Upload vers le serveur FTP")
    print("Nom du fichier à upload : (chemin possible)")

    nom_fichier = input()

    chemin_source = os.path.join(chemin, nom_fichier)

    if not os.path.exists(chemin_source):
        print(f"Erreur : Le fichier '{nom_fichier}' existe pas")
        return

    if not os.path.isfile(chemin_source):
        print("Ce n'est pas un fichier !")
        return

    nom_fichieronly = os.path.basename(chemin_source)
    nom_sans_extension, extension = os.path.splitext(nom_fichieronly)

    region_lower = region.lower()
    cheminftp = os.path.join("serveurftp_paris", region_lower)

    num_v = 1
```

Pour mieux comprendre cette fonction, il faut comprendre les éléments utilisés :

os.path.basename() qui récupère le nom du fichier, sans le chemin.

os.path.splitext() qui séparera ici le nom et l'extenstion du fichier.

os.path.isfile() qui vérifie si c'est un fichier et pas un dossier.

os.path.exists() qui vérifie si l'élément existe déjà.

Donc là on fait des vérifications.

On garde num_v pour le numéro de version.

```python
if os.path.exists(cheminftp):
    fichiers = os.listdir(cheminftp)

    version_actuelles = []
    for fichier in fichiers:

        if fichier.startswith(f"{nom_sans_extension}_V") and fichier.endswith(extension):
            try:
                partie_v = fichier.replace(f"{nom_sans_extension}_V", "").replace(extension, "")
                num = int(partie_v)
                version_actuelles.append(num)
            except ValueError:
                pass

    if len(version_actuelles) > 0:
        num_v = max(version_actuelles) + 1
```

Verification si le cheminftp existe.

Ensuite on os.listdir(cheminftp) : On retourne une liste de tous les fichiers et dossiers dans cheminftp.

Créer une liste vide pour stocker les numéros de versions.

Ensuite, une boucle pour vérifier si le fichier commence par le nom et _V, puis on cherche les fichiers commençant par cette nomination ET (parce qu'il y a le and) on cherche la même chose pour l'extension.

Ensuite, try pour gérer les erreurs :

On remplace le nom_sans_extension+_V par "" qui signifie rien/vide.

Pareil pour l'extension.

Et on convertit le résultat (le nombre qui est en string) en nombre entier avec int.

Ca va nous servir pour faire des calculs avec ce nombre.

Append permet d'ajouter num à version_actuelles, afin de lister avec les autres versions.

Le pass c'est pour éviter que ça s'arrête là.

On ajoute à la valeur maximale de la liste version_actuelles + 1 comme ça on a la nouvelle version.

```python
nom_fichier_v = f"{nom_sans_extension}_V{num_v}{extension}"

chemin_destination = os.path.join(cheminftp, nom_fichier_v)

try:
    shutil.copy(chemin_source, chemin_destination)
    print("-----------------------------------------------")
    print('--------------------------------------------')
    print(f"Version créée : V{num_v}")

except Exception as e:
    print(f"Erreur lors de l'upload : {e}")
```

Et derrière on construit le tout nouveau fichier !

Fonctionnement :

La navigation fonctionne de cette façon :

On voit avec une tabulation quel fichier est dans quel dossier, et ainsi de suite...

Conseil d'utilisation : Mettre l'arborescence à chaque fois pour voir ce que tu veux faire

Modifier un fichier :

Pas de / au début ! (Il fait : fichier_region/)

On peut voir le contenu du fichier avant la modification, et après la modification

On doit mettre le nom du nouveau fichier après l'endroit où l'on veut copier.

Copier un élément :

On doit mettre le nom du nouveau fichier après l'endroit où l'on veut copier.

```
Modification dans la region de Marseille.
Nom de l'élement à copier ( chemin possible oui oui)
achanger/coucou.txt
Nom de la destination (avec chemin si besoin)
achanger/coucou2.txt
Fichier copié : fichiers_marseille/achanger/coucou.txt vers fichiers_marseille/achanger/coucou2.txt
Appuyez sur Entrée pour continuer...
```

Déplacer un élément :

Le déplacement agit de la même façon que la copie, sauf que derrière ça efface. Donc on met aussi le nom du dossier qu'on veut.

```
Votre choix 1, 2 ou 3 : 1
Déplacement dans la région de Marseille.
Nom de l'élément à déplacer :
dossier.txt
Nom de la destination :
test/dossier.txt
Element deplacé !
Appuyez sur Entrée pour continuer...
```

Supprimer un élément :

La suppression agit sur les fichiers ou les dossiers. Si tu supprimes un dossier, ça supprime tout dedans également, d'où le message de confirmation.

Pas de / au début je rappelle.

```
Votre choix 1, 2 ou 3 : 1
Suppression dans la région de Marseille.
ATTENTION, supprimé c'est supprimé reprendre c'est pas possible !
Nom de l'élément à supprimer :
arbre/
Dossier à supprimer : fichiers_marseille/arbre/
Êtes-vous sûr de supprimer ??? Tapez oui.oui
Dossier supprimé : fichiers_marseille/arbre/
Appuyez sur Entrée pour continuer...
------------------------
```

Upload d'un fichier vers le serveur FTP :

Mettre le nom d'un fichier appartenant affichée, et choisir un dossier où le placer.

```
------------------------------------------------------------
Votre choix : 16
Upload d'un élément...
Dans quelle région voulez-vous upload ?
1 - Marseille
2 - Rennes
3 - Grenoble
Votre choix 1, 2 ou 3 : 1
Upload dans la région de Marseille.
Upload vers le serveur FTP
Nom du fichier à upload : (chemin possible)
cbn.txt
-----------------------------------------------
-------------------------------------------
Version créée : V2
Appuyez sur Entrée pour continuer...
```

Téléchargement d'un fichier vers local :

Même utilisation, et ENTREE si on veut le mettre à la racine.

```
3 - Grenoble
Votre choix 1, 2 ou 3 : 1
Téléchargement dans la région de Marseille.
Download depuis le serveur FTP Paris
Fichiers disponibles :

1. cbn_V1.txt
2. cbn_V2.txt
-------------------
Tapez le nom complet du fichier à télécharger :
cbn_V2.txt
____________________________________________________
Choisissez où télécharger le fichier cbn_V2.txt :
cbon/cbn_V2.txt
-----------------------------------------------
-------------------------------------------
Fichier téléchargé avec succès !
Appuyez sur Entrée pour continuer...
-------------------------
```